Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 9

Disciplina: Tehnici de programare

Tema: ”*Colecții: liste, tupluri, mulțimi, dicționare.*”

A efectuat : st. gr.SI – 262 Caragia Denis

A verificat : asistent universitar Emil Cebotaroș

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea are ca scop folosirea colecțiilor din Python (lista, tuplul, mulțimea și dicționarul) și a comprehensions pentru a prelucra date reale. Alegerea colecției depinde de ce trebuie făcut cu datele: lista păstrează ordinea și dublurile, mulțimea scoate dublurile și permite diferența dintre două seturi, dicționarul grupează câmpuri cu nume, iar tuplul fixează o pereche de valori. Situația practică este enumerarea unei mașini Linux compromise, în drumul spre drepturi de root.

Lucrarea are două părți. În Lucrarea A, programul enum.py citește trei capturi (suid.txt, sudo.txt, crontab.txt), grupează constatările pe categorii și scrie un raport JSON. În Lucrarea B, programul diferenta.py compară fișierele SUID de acum cu cele ale unui sistem curat (baza_suid.txt) și scoate ce a apărut nou și periculos. Datele provin de la o mașină de laborator izolată, nu de la un sistem real. Fișierele se află în folderul Arsenal/privesc din depozitul Work. Raportul descrie pașii în ordinea executării, fiecare cu captura de ecran corespunzătoare.

DESFĂȘURAREA LUCRĂRII

Lucrarea A. enum.py: adună și grupează

Prima listă

Funcția citeste_linii deschide un fișier, curăță fiecare linie cu strip() și păstrează doar liniile care nu sunt goale și nu încep cu #. Pentru suid.txt rezultă 14 linii. Sunt mai multe decât căile distincte, pentru că lista păstrează tot, în ordine, inclusiv dublura: /usr/bin/passwd apare de două ori.

Nume unice: de la listă la mulțime

Ca să decid dacă un fișier este periculos, am nevoie de numele lui, nu de toată calea. Funcția nume întoarce ce se află după ultimul „/”, cu rsplit("/", 1)[-1]. Numele le-am adunat într-un set comprehension, adică într-o mulțime, care păstrează fiecare valoare o singură dată. Din 14 linii au rămas 13 nume distincte, deci mulțimea a aruncat dublura.

Constatările SUID: listă de dicționare

Fiecare fișier SUID a devenit o fișă cu trei câmpuri: calea, numele și riscul. Funcția suid_constatari scoate dublurile cu set(), sortează căile cu sorted() și construiește, cu un list comprehension, un dicționar pentru fiecare cale. Riscul este „mare” dacă numele se află în GTFO, o mulțime de binare cunoscute ca scăpări spre root (find, vim, nmap, cp, bash și altele), și „normal” în rest. Rezultatul este o listă cu 13 constatări, dintre care 4 au risc mare: cp, find, nmap și vim.basic.

Sudo și cron

Pentru sudo.txt am tăiat fiecare linie la primul „ : ” cu split(" : ", 1) și am despachetat cele două părți în user și drept. Riscul este „mare” dacă dreptul conține NOPASSWD, adică se poate rula comanda fără parolă. Din cele 3 reguli, 2 au risc mare: ana cu vim.basic și www-data cu find. Pentru crontab.txt am împărțit linia în câmpuri cu split(). Primele cinci câmpuri sunt programul, câmpul cu indexul 5 este userul, iar de la indexul 6 începe comanda. Rândul de comentariu a fost eliminat încă de citeste_linii, iar rezultatul are 3 sarcini: două rulează ca root și una ca www-data.

Funcția enumereaza adună totul într-un dicționar, în care cheia este categoria (suid, sudo, cron), iar valoarea este lista constatărilor ei. Verificarea arată 13, 3 și 3 constatări.

Raportul și fișierul JSON

Pentru a face programul utilizabil din linia de comandă am folosit argparse: folderul cu capturi este un argument obligatoriu, iar --json este un comutator care afișează raportul ca JSON. Fără comutator, programul afișează pe fiecare categorie câte constatări a găsit și câte au risc mare. La cron apare 0, deoarece dicționarele de cron nu au cheia „risc”, iar citirea cu .get("risc") întoarce None în loc să dea eroare. Utilizatorii implicați i-am obținut prin reuniunea a două mulțimi, cu operatorul |: ana, root, www-data. La final, raportul este salvat în enum.json cu json.dump.

Fișierul se numește enum.py, ca în sarcină, dar numele coincide cu modulul enum din biblioteca standard Python. Rulat direct din folderul lui, programul dădea eroare de import

circular. De aceea l-am rulat ca modul, din folderul Work: python -m Arsenal.privesc.enum Arsenal/privesc/start.

```
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

Lucrarea B. Ce e nou față de un sistem curat

Două mulțimi: curat și acum

În diferenta.py am importat citeste_linii din enum.py, ca să nu scriu citirea a doua oară. Funcția cai întoarce mulțimea căilor dintr-un fișier. Sistemul curat (baza_suid.txt) are 9 căi, iar sistemul de acum (suid.txt) are 13, adică 14 linii minus dublura.

Diferența: ce a apărut

Fișierele SUID noi sunt cele din „acum” care nu se află în „baza”. Am folosit un singur operator, scăderea de mulțimi (acum - baza), în locul unei bucle duble. Au rezultat 4 fișiere noi, afișate în ordine alfabetică: /bin/cp, /usr/bin/find, /usr/bin/nmap și /usr/bin/vim.basic.

Dicționarul de risc și filtrul final

Pentru fișierele noi am construit, cu un dict comprehension, o fișă nume - risc, folosind nume și GTFO importate din enum.py. Apoi, cu un list comprehension cu if, am păstrat doar căile cu risc mare. Toate cele patru fișiere apărute sunt periculoase: cineva a pus bit SUID pe find, nmap, cp și vim, iar oricare dintre ele poate da acces de root. O asemenea constatare trebuie scoasă în față, nu ascunsă în raport.

Testul cu pytest

Am scris în test_diferenta.py un test mic, cu date introduse de mână, care verifică ideea de bază: din {"a", "b", "c"} fără {"a", "b"} rămâne doar {"c"}. Testul nu citește fișiere și nu are nevoie de rețea. Rulat cu pytest, trece.

```
PS C:\Users\carag\OneDrive\Desktop\uni\Work> python -m pytest -q arsenal/privesc/test_diferenta.py
.
1 passed in 0.04s
```

CONCLUZIE

În această lucrare am folosit toate cele patru colecții din Python în locul potrivit. Lista a păstrat liniile citite și constatările, în ordine și cu dubluri. Mulțimea a scos dublurile și mi-a dat diferența dintre sistemul de acum și cel curat dintr-un singur operator. Dicționarul a grupat câmpurile unei constatări și categoriile raportului, iar tuplul a despachetat linia din sudo în user și drept. Comprehensions au făcut codul scurt, iar .get a evitat erorile acolo unde o cheie poate lipsi. Rezultatul este o unealtă care enumeră o mașină, scrie un raport JSON valid și arată imediat ce s-a schimbat și este periculos: patru fișiere SUID noi, toate cu risc mare. Cea mai utilă idee reținută este că alegerea colecției potrivite face programul mai simplu și mai greu de greșit.